SHOPSPHERE ENGINEERING

OPERATIONS RUNBOOK

Payment Service Deployment Runbook

Deploying payment-service to production, watching the rollout, safe diagnostics and rollback

Document number	SS-RB-PAY-031
Runbook version	3.1
Last reviewed	2026-07-30
Owner	SRE Team
Service	payment-service (production)
Use when	Deploying payment-service, or when payment errors rise after a deploy
Safety	All checks in Section 4 are read-only

Fictional document. ShopSphere, its services, people and data are synthetic. This document is part of the TraceIQ training dataset and is not documentation for any real system.

Contents

1. Scope . . . 3
1.1 Roles . . . 3
1.2 How a payment-service Deploy Works . . . 3

2. Pre-Deployment Checklist . . . 4
2.1 Notes on Each Check . . . 4

3. Post-Deployment Monitoring . . . 4
3.1 What to Watch . . . 5
3.2 The 15-Minute Watch Window . . . 5

4. Safe Diagnostic Checks (read-only) . . . 5
4.1 D1 - Group Errors by Code and Version . . . 6
4.2 D2 - Compare a Failing and a Succeeding Request . . . 6
4.3 D3 - Look Up the Error Code . . . 6
4.4 D4 - Read the Running Config . . . 6
4.5 D5 - Reproduce in Staging . . . 6
4.6 D6 - Check Data Stores . . . 7
4.7 What Each Check Cannot Tell You . . . 7

5. Rollback Considerations . . . 7
5.1 Decision Flow . . . 8
5.2 Rollback Steps . . . 8
5.3 Confirming Recovery . . . 8

6. Do Not . . . 8

7. Communication . . . 9

8. Glossary and Revision History . . . 9

Appendix A. Go/No-Go Record . . . 10

Appendix B. Diagnostic Worksheet . . . 10

Appendix C. Handover Note Template . . . 11

1. Scope

Use this runbook when deploying payment-service to production and when payment errors rise after a deploy. Everything in Section 4 is read-only and safe to run during an incident.

1.1 Roles

Four roles take part in a payment-service deploy. One person can hold more than one role, but the incident commander must be named before a rollback is approved.

Role	Responsibilities in this runbook
Release manager	Runs the go/no-go meeting. Makes sure the release notes are reviewed (P1) and that on-call SRE knows the deploy window (P5).
Service owner	Confirms staging smoke tests passed (P2), that every client sends request fields the new version accepts (P3), and whether the release has database migrations (P4).
On-call SRE	Watches the payment dashboard for 15 minutes after the rollout completes (Section 3). Runs the safe diagnostic checks (Section 4).
Incident commander	Approves a rollback (Section 5).

Table 1-1. Roles.

1.2 How a payment-service Deploy Works

payment-service is deployed by the deploy pipeline. The pipeline does a rolling deploy across the four production instances: one instance is moved to the new version at a time while the others keep serving traffic. When every instance runs the new version, the pipeline checks GET /healthz on each instance.

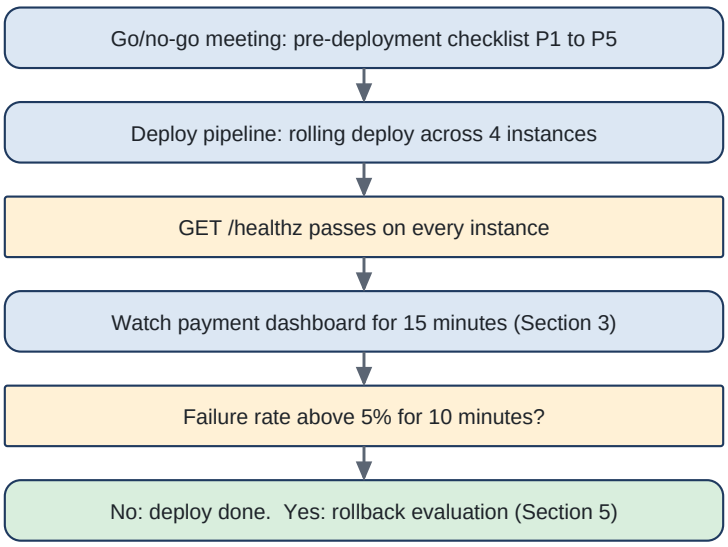


Figure 1-1. Lifecycle of a payment-service production deploy.

2. Pre-Deployment Checklist

Complete every check before the deploy starts. Record the result of each check in the go/no-go meeting.

#	Check	Who
P1	Release notes reviewed in the go/no-go meeting.	Release manager
P2	Staging smoke tests passed.	Service owner
P3	Confirm every client sends request fields that the new version accepts (check dependency upgrades too).	Service owner
P4	Confirm whether the release contains database migrations.	Service owner
P5	On-call SRE is aware of the deploy window.	Release manager

Table 2-1. Pre-deployment checklist.

2.1 Notes on Each Check

#	How to do it	Why it matters
P1	Read every item in the release notes aloud or on screen in the meeting.	Changes that are not discussed are easy to miss.
P2	Look at the latest staging smoke test run for the exact build that will be deployed.	A passing run for a different build proves nothing.
P3	List every client of payment-service. For each one, compare the fields it sends with the fields the new version accepts. If the release upgrades a dependency that reads requests, read that dependency's changelog too.	A version can reject requests that an older version accepted, even if the API document did not change.
P4	Check the release for database migrations.	Migrations change how safe a rollback is (Section 5).
P5	Tell on-call SRE the start time of the deploy window.	Someone must watch the dashboard after the rollout.

Table 2-2. How to complete each check.

> **Smoke tests are not client tests.** Check P2 only covers the requests that the smoke test suite sends. Check P3 is how you cover the requests real clients send.

3. Post-Deployment Monitoring

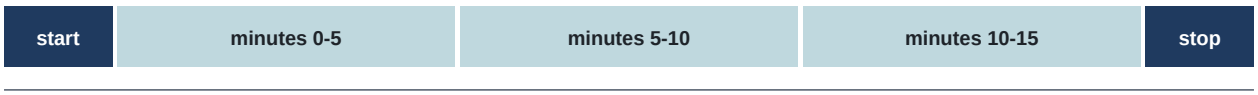
Watch the payment dashboard for 15 minutes after the rollout completes. The /healthz check only proves the process is up; it does not send payment requests. If the payment failure rate stays above **5% for 10 minutes**, start rollback evaluation (Section 5).

3.1 What to Watch

Signal	Where	Normal	Act when
Payment failure rate	Payment dashboard	Close to 0%	Above 5% for 10 minutes: start rollback evaluation
Alert PaymentFailureRateHigh	Alerting	Not firing	Firing (failure rate above 5% for 5 minutes): start Section 4 checks
payment-service latency p95	Payment dashboard	Stable compared with before the deploy	Clear change after the deploy: note it and check Section 4
order-service payment-call latency	Payment dashboard	Stable compared with before the deploy	Clear change after the deploy: note it and check Section 4

Table 3-1. Signals to watch during the 15-minute window.

3.2 The 15-Minute Watch Window



watch starts when the rollout completes; 5-minute metric buckets

Figure 3-1. Watch window after the rollout. Metrics arrive in 5-minute buckets.

Metrics are exported in 5-minute buckets, so the window covers three complete buckets. A failure rate that stays above 5% for two consecutive buckets (10 minutes) meets the rollback evaluation threshold.

> **Important.** A passing /healthz check does not mean payments work. It only shows that the process is running.

4. Safe Diagnostic Checks (read-only)

Run these checks when payment errors rise after a deploy. None of them changes production. Run them in order; each one narrows down the next.

#	Check	What it tells you
D1	Group recent errors by error_code and service version.	Whether errors started with the new version.
D2	Compare one failing request with one succeeding request (fields present, client flow).	Whether only some request shapes fail.
D3	Look up the error_code in the Payment API Specification.	What the error actually means. Do not guess from the number.
D4	Read the running config with GET /internal/config (read-only).	Whether a setting differs between versions.
D5	Reproduce in staging with the same request body.	Confirms the failure without touching production.
D6	Check data-store latency (redis, postgres) on the infra dashboard.	Rules data stores in or out.

Table 4-1. Safe diagnostic checks.

4.1 D1 - Group Errors by Code and Version

Search the structured logs of payment-service for the period since the deploy started. Count records by error_code and by version. If one error code appears only on the new version, the deploy is the first suspect.

D1 example

```
# illustrative query - read-only
logs search service=payment-service severity=ERROR since=<deploy start>
     | count by error_code, version
```

4.2 D2 - Compare a Failing and a Succeeding Request

Pick one request that failed and one that succeeded in the same period. Compare the fields that were present and the client flow that sent each one. If only one request shape fails, the cause is usually that shape, not the network or a data store.

Compare	Failing request	Succeeding request
Client service and flow		
Fields present		
payment-service version		
HTTP status and error_code		

Table 4-2. Worksheet for D2.

4.3 D3 - Look Up the Error Code

Find the error_code in the error table of the Payment API Specification and read its HTTP status and meaning there. Do not guess the meaning from the number in the code.

4.4 D4 - Read the Running Config

GET /internal/config returns the configuration the running instance is using. It is read-only. Compare the output of an instance on the new version with the output you expect from the previous version.

D4 example

```
# illustrative request - read-only
GET /internal/config      (on one payment-service instance)
```

4.5 D5 - Reproduce in Staging

Send the same request body that failed in production to staging, once on the new version and once on the previous version. Use made-up test values for any token. If the new version fails and the previous one succeeds, the failure is confirmed without touching production.

4.6 D6 - Check Data Stores

Open the infra dashboard and read the latency of redis and postgres for the same period. If both are normal, the data stores can be ruled out.

4.7 What Each Check Cannot Tell You

Each check answers one narrow question. Use the table to avoid reading more into a result than it supports.

#	It can show	It cannot show
D1	Which error codes appear, on which version, and since when.	Why the error happens.
D2	Which request shape or client flow fails.	Which code change made that shape fail.
D3	The documented meaning and HTTP status of an error code.	Whether the running version behaves as documented.
D4	The configuration the running instance uses.	What a setting would do if it were changed.
D5	Whether the failure repeats outside production.	Whether a proposed fix works, unless you test the fix in staging too.
D6	Whether redis and postgres were slow.	Anything about request content.

Table 4-3. Limits of each diagnostic check.

> **Keep a record.** Write down which checks you ran and what each one showed. Mark every statement as verified or not verified. Appendix B gives a worksheet.

5. Rollback Considerations

Rollback is done by re-running the deploy pipeline with the previous version. It needs approval from the incident commander. Before rolling back, check that the release has no database migrations (if it does, contact the Payments Team first). After rollback, confirm recovery on the dashboard for at least 5 minutes.

5.1 Decision Flow

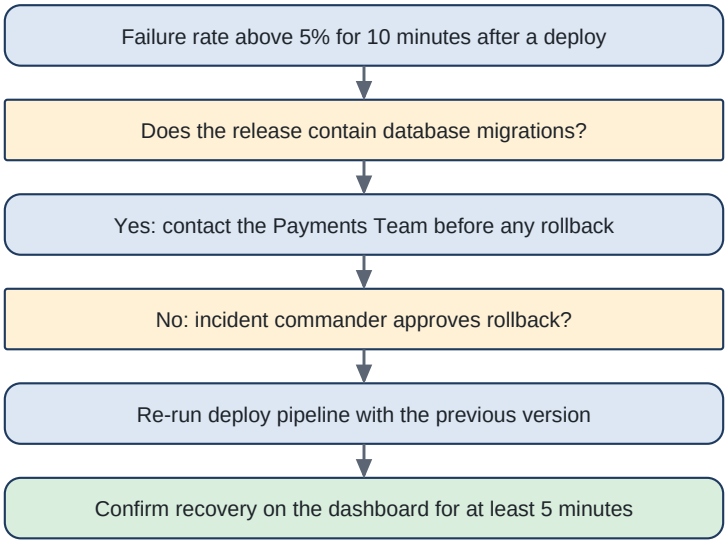


Figure 5-1. Rollback decision flow.

5.2 Rollback Steps

1. Record the current version and the previous version in the incident channel.
2. Check the release notes for database migrations. If there are any, contact the Payments Team first.
3. Get approval from the incident commander.
4. Re-run the deploy pipeline with the previous version.
5. Wait until every instance runs the previous version.
6. Confirm recovery on the dashboard for at least 5 minutes (Section 5.3).

5.3 Confirming Recovery

Signal	Recovered when
Payment failure rate	Back to the level before the deploy
Alert PaymentFailureRateHigh	Resolved
Latency (payment-service, order-service payment calls)	Back to the level before the deploy

6. Do Not

Do not edit production config by hand. Do not restart redis or postgres to 'see if it helps'. Do not replay failed payments without their original idempotency keys.

Do not	Why
Edit production config by hand	Changes are not reviewed and are hard to undo or trace.
Restart redis or postgres to 'see if it helps'	Restarting a data store affects every service that uses it.
Replay failed payments without their original idempotency keys	Without the original key a payment can be recorded twice.

Table 6-1. Actions that are not allowed during a payment incident.

7. Communication

When	What to do
Before the deploy	Release manager confirms P5: on-call SRE knows the deploy window.
Alert fires or failure rate rises	On-call SRE starts the Section 4 checks and opens an incident if payments are failing.
Incident opened	Name an incident commander. Page the payment-service owners.
Rollback needed	Incident commander approves it (Section 5).
Shift change	Write a handover note: status, what is verified, what is still open, next steps.

Table 7-1. Who communicates what.

8. Glossary and Revision History

Term	Meaning
Deploy pipeline	The automated tool that rolls out and rolls back service versions.
Go/no-go meeting	Meeting before a deploy where the checklist in Section 2 is completed.
Incident commander	The person who leads an incident and approves a rollback.
Read-only check	A check that reads data or configuration and changes nothing.
Rolling deploy	Updating instances one at a time while the others keep serving traffic.
Smoke tests	A small automated test suite run in staging before a deploy.

Version	Date	Change
3.1	2026-07-30	Last review (this version).

Appendix A. Go/No-Go Record

Fill in this record during the go/no-go meeting (Section 2). Keep it with the deploy pipeline run.

Item	Value
Service	payment-service
Current version	
New version	
Planned start (UTC)	
Deploy pipeline run	
P1 Release notes reviewed	yes / no
P2 Staging smoke tests passed	yes / no
P3 All clients send fields the new version accepts	yes / no - list clients checked:
P4 Database migrations	none / yes - details:
P5 On-call SRE aware	yes / no
Decision	go / no-go
Approved by (name, time UTC)	

Table A-1. Go/no-go record template.

Appendix B. Diagnostic Worksheet

Use one row per check from Section 4. Mark each finding as verified or not verified.

#	Ran at (UTC)	Result	Verified?
D1			yes / no
D2			yes / no
D3			yes / no
D4			yes / no
D5			yes / no
D6			yes / no

Table B-1. Diagnostic worksheet template.

Appendix C. Handover Note Template

At a shift change during an incident, write a handover note with these parts.

Part	What to write
Header	Incident ID, from, to, time written (UTC).
STATUS	Is the incident ongoing or mitigated? Which version is running?
VERIFIED	Facts confirmed by a check, with the check that confirmed them.
NOT VERIFIED / STILL OPEN	Hypotheses and questions nobody has confirmed yet.
NEXT STEPS	Numbered actions with an owner.

Table C-1. Handover note template.